МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский ядерный университет «МИФИ»
(НИЯУ МИФИ)

ИНСТИТУТ ИНТЕЛЛЕКТУАЛЬНЫХ КИБЕРНЕТИЧЕСКИХ СИСТЕМ

КАФЕДРА «КОМПЬЮТЕРНЫЕ СИСТЕМЫ И ТЕХНОЛОГИИ» (№12)

ОТЧЕТ
КУРСОВОЙ ПРОЕКТ (СХЕМОТЕХНИКА ЦИФРОВЫХ УСТРОЙСТВ)

Семестр 5

Тема: «Сортировщик продукции на конвейере»

Студенты Коротаев С. А., Моря К. О. Группа С24-501
ФИО

Руководитель Решетько В. М.
ФИО, должность, звание, степень

Студент _____ Коротаев С. А.
подпись ФИО

Студент _____ Моря К. О.
подпись ФИО

Руководитель _____ Решетько В. М.
подпись ФИО

Оценка _____

Баллы, ECTS _____

«____»_____2026 г.

Москва 2026 г.

ОГЛАВЛЕНИЕ

1. Введение 3
2. Основные функции 3
3. Условное графическое обозначение и список портов ввода-вывода 3
4. Внутренняя структура устройства 4
4.1. Ядро сортировщика 4
4.2. Обёртка внешних датчиков 4
4.3. Демонстрационный генератор 5
4.4. Верхний уровень платы 5
4.5. Тестбенч 6
5. Принцип работы 6
6. Тактовая организация 8
7. Тестирование 8
8. Компиляция и синтез схемы 9
9. Результаты прошивки ПЛИС 11
10. Ограничения и допущения 11
11. Заключение 12
12. Приложение 13

1. Введение

Сортировщик определяет пригодность изделия по восьмиразрядному коду веса. Два датчика отмечают прохождение зоны измерения. Автомат фиксирует отсчёт и формирует импульс отбраковки, если вес выходит за допустимый диапазон.

2. Основные функции

1. Обнаружение изделия по любому из двух датчиков.
2. Фиксация веса после прохождения центра.
3. Проверка диапазона 95–105 включительно.
4. Отбраковка недопустимого изделия импульсом заданной длительности.
5. Ожидание освобождения датчиков перед следующим изделием.
6. Возврат в исходное состояние по активному низкому сбросу.

Разрешение конвейера постоянно равно единице, включая сброс и отбраковку.

3. Условное графическое обозначение и список портов ввода-вывода

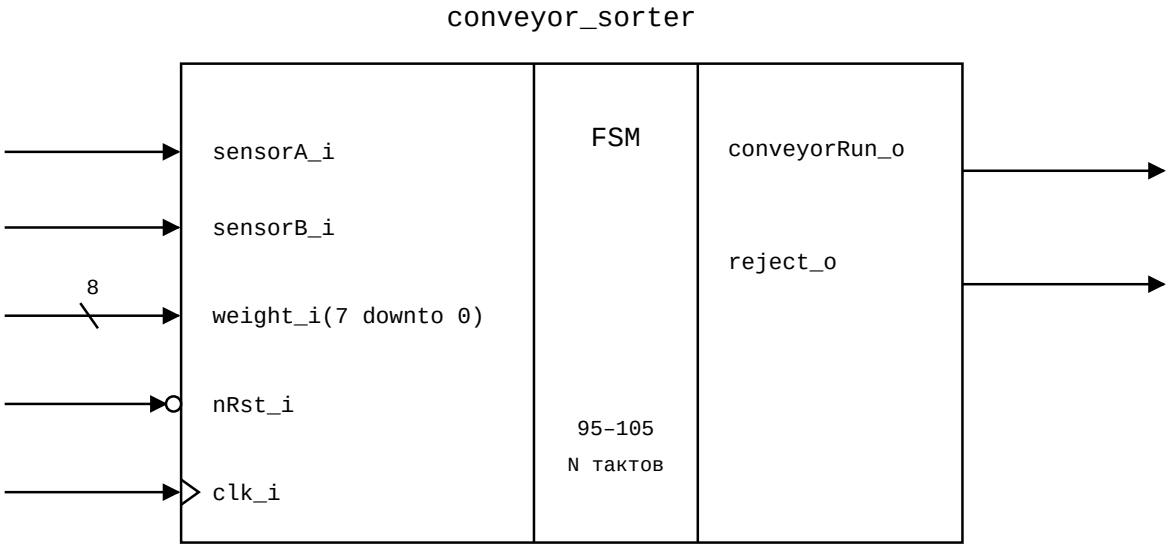


Рис. 1. УГО модуля conveyor_sorter

Таблица 1. Порты ядра

Сигнал	Направление	Разрядность	Назначение
clk_i	in	1	Положительный фронт такта
nRst_i	in	1	Асинхронный сброс при нуле
sensorA_i	in	1	Первый датчик; активен при 1
sensorB_i	in	1	Второй датчик; активен при 1
weight_i	in	8	Беззнаковый код (7 downto 0)
conveyorRun_o	out	1	Постоянное разрешение конвейера
reject_o	out	1	Импульс отбраковки

Треугольник обозначает тактовый вход, работающий по положительному фронту; кружок — активный низкий сброс; косая черта с числом 8 показывает ширину шины веса.

4. Внутренняя структура устройства

Ядро, генератор воздействий и верхний модуль используют общий такт. Состояние автомата и данные обновляются на положительном фронте.

4.1. Ядро сортировщика

conveyor_sorter.vhd содержит конечный автомат с пятью состояниями. Переходы заданы через case state_r в отдельном тактовом процессе; регистры данных и выходы обновляются в отдельных процессах. Промежуточного сигнала nextState_s нет. Допустимый диапазон веса — 95–105 включительно; REJECT_PULSE_CYCLES = 5_000_000.

Таблица 2. Регистры ядра

Сигнал	Тип	Назначение
state_r	state_type	Текущее состояние
previousWeight_r	8 бит	Предыдущий отсчёт
measuredWeight_r	8 бит	Зафиксированный вес
centerFlag_r	1 бит	Признак прохождения центра
rejectCount_r	natural	Счётчик 0 … N − 1

4.2. Обёртка внешних датчиков

conveyor_sorter_top.vhd синхронизирует датчики двумя регистрами. Сброс устанавливается асинхронно и снимается через resetSync_r. Вес проходит через weightMeta_r и weightSync_r; weightStable_r обновляется при совпадении двух последовательных синхронизированных отсчётов. Совпадение отсчётов позволяет отсеять кратковременные изменения. При подключении внешних весов

согласованность передачи всех восьми разрядов нужно обеспечивать отдельно. В демонстрационной сборке данная обёртка не используется.

4.3. Демонстрационный генератор

`flow_tester.vhd` меняет позицию 0 … 15 раз в STEP_CYCLES = 5_000_000 тактов. При 50 МГц шаг длится 100 мс, одно изделие — 1,6 с. Флаг `rejectProduct_r` меняется при переходе позиции 15 → 0: допустимый профиль с весом 100 чередуется с недопустимым весом 80. После сброса флаг равен нулю, первым идёт допустимое изделие. В неуказанных позициях датчики и вес равны нулю.

Таблица 3. Активные позиции двух профилей

Позиция	A	B	Вес: допуст.	Вес: брак	Этап
2	1	0	20	20	Вход
3	1	0	50	30	Рост
4	1	1	80	60	Центр
5	1	1	100	80	Верхний отсчёт
6	1	1	98	78	Фиксация 100 / 80
7	0	1	70	50	Выход
8	0	1	30	30	Завершение

4.4. Верхний уровень платы

`qmtech_demo_top.vhd` соединяет генератор и ядро. Такт и активный низкий сброс подаются обоим модулям. Датчики и вес формируются внутри ПЛИС. Светодиодные выходы инвертированы: conveyorLed_n_o = not conveyorRun_s, rejectLed_n_o = not reject_s. Поэтому команда 1 в ядре даёт 0 на активном низком выходе индикации.

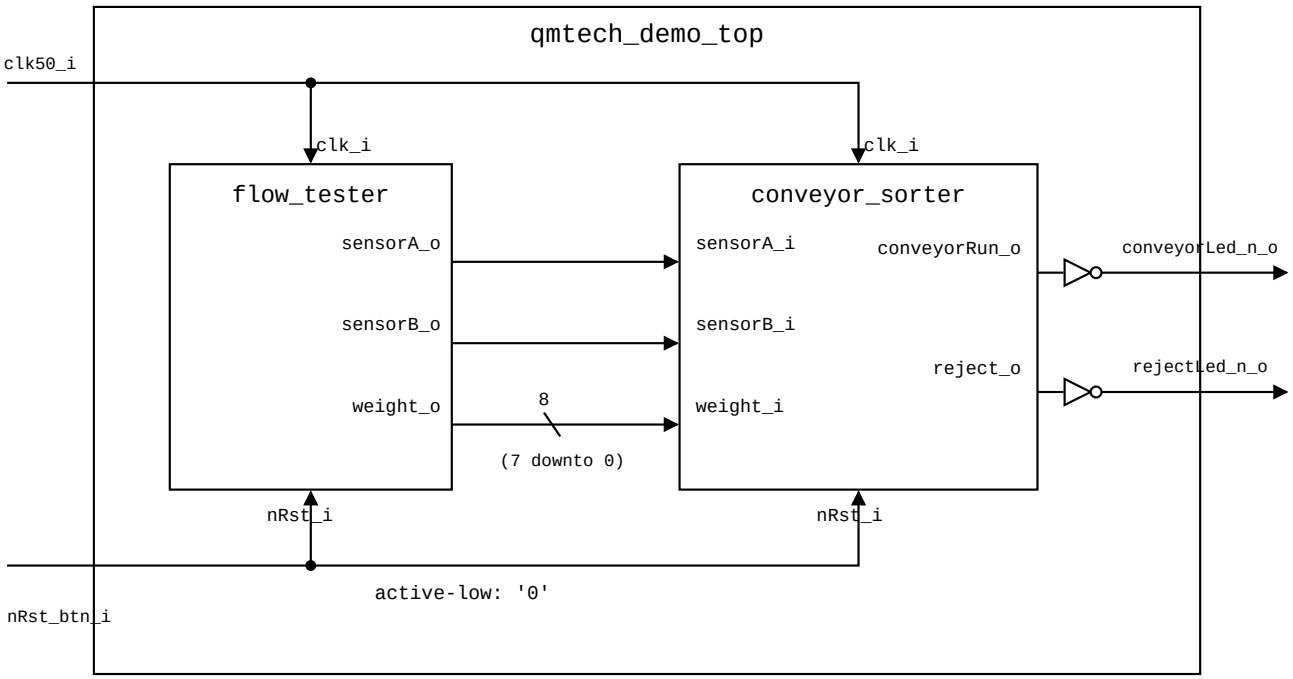


Рис. 2. Структура демонстрационного верхнего уровня

4.5. Тестбенч

`struct_tb.vhd` проверяет пять состояний автомата, границы весового диапазона, импульс отбраковки, ожидание освобождения датчиков, незавершённое измерение без обнаружения центра и асинхронный сброс во время отбраковки. Для теста REJECT_PULSE_CYCLES = 3. `flow_tester_tb.vhd` проверяет последовательность генератора совместно с ядром; `test_alternating.do` запускает оба сценария.

5. Принцип работы

При nRst_i = '0' устанавливается WAIT_STATE; регистры веса, флаг центра и счётчик обнуляются. В WAIT_STATE `previousWeight_r` получает текущий вес. Любой активный датчик переводит автомат в MEASURE_STATE.

В MEASURE_STATE совместная активность датчиков устанавливает `centerFlag_r`. При первом спаде веса после отмеченного центра либо при спаде во время совместной активности датчиков фиксируется `previousWeight_r`. Если оба датчика освободились после центра без спада, фиксируется тот же предыдущий отсчёт. До фиксации `previousWeight_r` обновляется на каждом фронте. `measuredWeight_r` получает отсчёт с предыдущего такта. Поэтому фиксируется значение перед первым спадом после центра, которое может отличаться от максимума за всё время прохождения изделия.

Обозначения: A и B — датчики; W — текущий вес; V — previousWeight_r; M — measuredWeight_r (весовые слова без знака); F — centerFlag_r; C — rejectCount_r; N — REJECT_PULSE_CYCLES.

P: W < V и (F = '1' или (A = '1' и B = '1')).
Q: A = '0' и B = '0' и F = '1'.

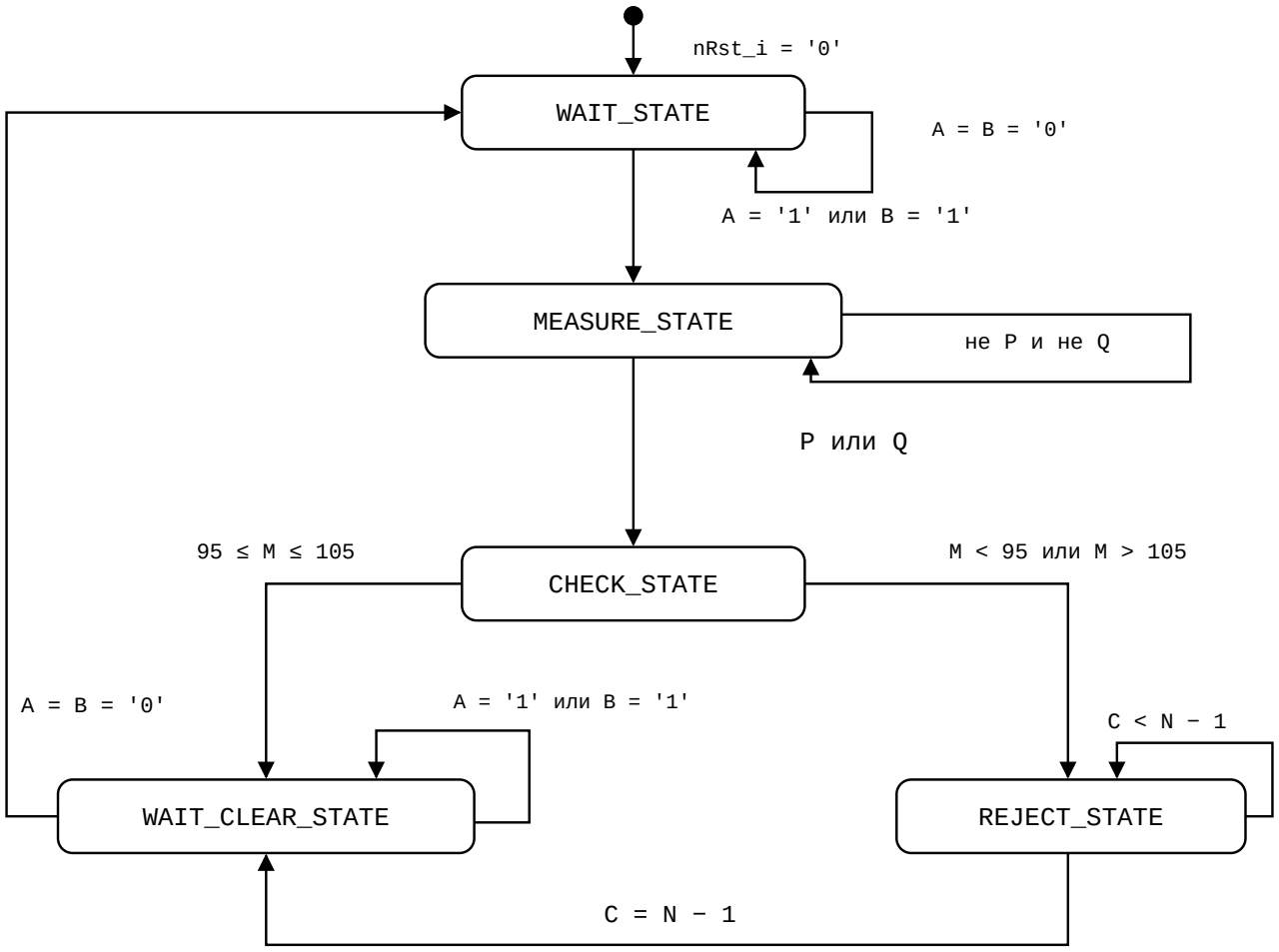


Рис. 3. Диаграмма состояний; сброс действует из любого состояния

Таблица 4. Условия переходов

Состояние	**Действия и переход**
WAIT_STATE	A = '1' или B = '1' → MEASURE_STATE; иначе ожидание. F ← 0, C ← 0, V ← W.
MEASURE_STATE	При P, иначе при Q: M ← V → CHECK_STATE. Иначе V ← W, состояние сохраняется. При A = B = '1' устанавливается F.
CHECK_STATE	95 ≤ M ≤ 105 → WAIT_CLEAR_STATE; иначе C ← 0 → REJECT_STATE.
REJECT_STATE	C = N − 1 → WAIT_CLEAR_STATE, C ← 0; иначе C ← C + 1, состояние сохраняется.
WAIT_CLEAR_STATE	A = B = '0' → WAIT_STATE; иначе ожидание. Данные сохраняются.

CHECK_STATE занимает один такт. reject_o = '1' только в REJECT_STATE, которое длится N тактов. После проверки и отбраковки автомат ждёт освобождения обоих датчиков. Асинхронный сброс имеет приоритет над тактом.

6. Тактовая организация

Номинальная частота такта — 50 МГц, период — 20 нс. Ядро и генератор работают от одного тактового сигнала; шаг генератора задаётся счётчиком без отдельного делённого такта.

Импульс отбраковки: 5_000_000 × 20 нс = 100 мс. В тесте: 3 × 20 нс = 60 нс. Параметр N положителен. Сброс ядра и генератора активный низкий, асинхронный. В демонстрации снятие кнопочного сброса не синхронизировано; синхронизатор предусмотрен в обёртке внешних датчиков. Частота Fmax не определена представленными материалами.

7. Тестирование

В исходнике тестбенча задано 12 проверок `assert` с уровнем `severity failure`. Сценарии продуктов покрывают нижнюю и верхнюю границы допустимого веса и значения по обе стороны диапазона. Также проверяются длительность импульса, ожидание освобождения датчиков, измерение без фиксации центра и асинхронный сброс при активной отбраковке.

Таблица 5. Проверяемые сценарии ядра

Сценарий	Входные условия	Ожидаемый результат
Вес ниже диапазона	94 и 80	Отбраковка; импульс 3 такта
Нижняя граница	95	Изделие проходит без отбраковки
Верхняя граница	105	Изделие проходит без отбраковки
Вес выше диапазона	106	Отбраковка; импульс 3 такта
Нет центра	Датчик A срабатывает без совместного обнаружения	Измерение не завершается
Сброс во время импульса	Асинхронный сброс при reject = 1	reject немедленно сбрасывается

Для совместной проверки генератора и сортировщика использован `flow_tester_tb.vhd`. В нём STEP_CYCLES = 8, REJECT_PULSE_CYCLES = 3, период такта — 20 нс. Один профиль изделия занимает 16 × 8 × 20 нс = 2,56 мкс; расчётная длительность импульса отбраковки — 60 нс. Ускорение позволяет просмотреть несколько изделий за один запуск.

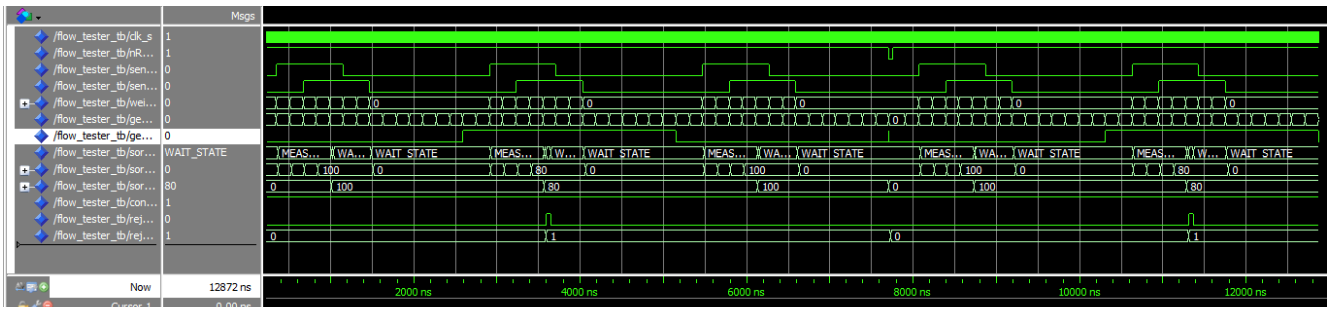


Рис. 4. Временная диаграмма совместной работы генератора и сортировщика, flow_tester_tb

На рисунке видны пять проходов изделия. В первых трёх зафиксированный вес равен 100, 80 и 100. При весе 100 сигнал отбраковки остаётся равным нулю; при весе 80 появляется импульс, а счётчик импульсов принимает значение 1. Между третьим и четвёртым проходами подан активный низкий сброс. Счётчик обнуляется, и последовательность начинается заново: сначала вес 100 без отбраковки, затем вес 80 с импульсом. Разрешение конвейера на всём показанном интервале равно единице.

Диаграмма иллюстрирует чередование профилей в предоставленной временной диаграмме. Масштаб снимка не позволяет измерить длительность короткого импульса. Код обновлённого тестбенча компилируется в ModelSim без ошибок и предупреждений, но запуск симуляции в рабочей среде завершился ошибкой RPC до начала теста. Поэтому успешное прохождение assert для текущего кода не подтверждено; изображение не является доказательством запуска обновлённых тестов.

8. Компиляция и синтез схемы

Приложенный снимок Flow Summary показывает успешную сборку предыдущей версии проекта в Quartus Prime Lite Edition 20.1.1 Build 720 для Cyclone 10 LP 10CL080YU484C8G: 119 логических элементов и 69 регистров. После исправления автомата и тестбенча Quartus отсутствовал в рабочей среде, поэтому эта сборка не подтверждает синтез текущих исходников и её показатели нельзя относить к ним.

Compilation Report - qmtech_demo

Flow Summary

<<Filter>>

Flow Status	Successful - Wed Oct 07 10:43:12 2026
Quartus Prime Version	20.1.1 Build 720 11/11/2020 SJ Lite Edition
Revision Name	qmtech_demo
Top-level Entity Name	qmtech_demo_top
Family	Cyclone 10 LP
Device	10CL080YU484C8G
Timing Models	Final
Total logic elements	119 / 81,264 (< 1 %)
Total registers	69
Total pins	4 / 290 (1 %)
Total virtual pins	0
Total memory bits	0 / 2,810,880 (0 %)
Embedded Multiplier 9-bit elements	0 / 488 (0 %)
Total PLLs	0 / 4 (0 %)

Рис. 5. Сводка успешной компиляции проекта qmtech_demo

Приведённые далее снимки RTL и технологических схем также относятся к предыдущей версии проекта. Они служат иллюстрацией структуры и не подтверждают результаты сборки текущих исходников.

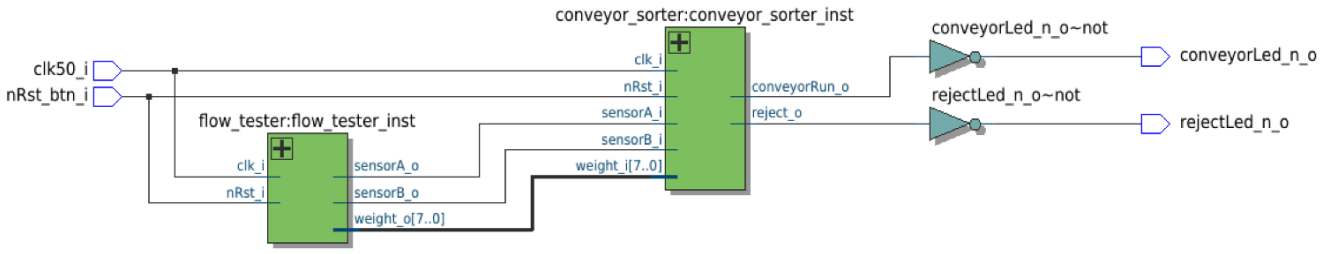


Рис. 6. RTL-схема: генератор, ядро и инверторы выходов

В RTL-схеме датчики и шина веса соединяют генератор с ядром. В технологической схеме постоянный выход conveyorLed_n_o оптимизирован до нуля, а rejectLed_n_o формируется инверсией признака REJECT_STATE.

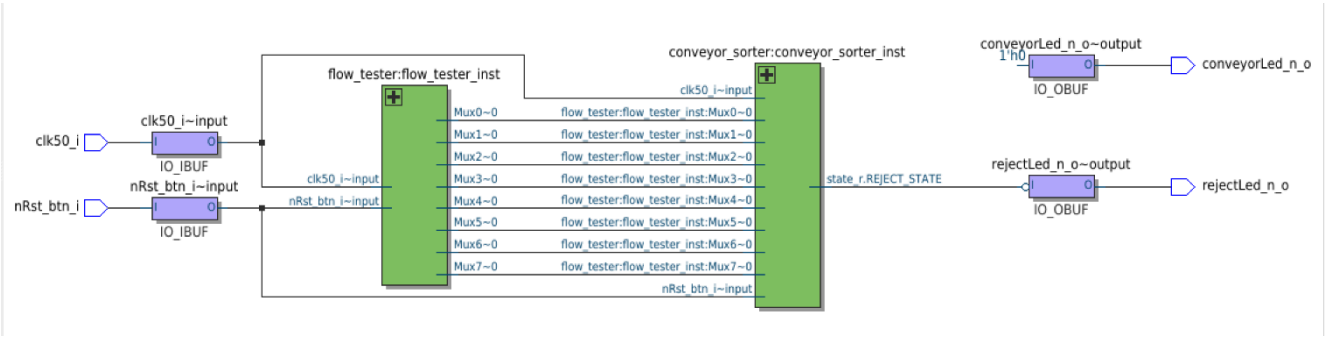


Рис. 7. Технологическая схема после синтеза (Post-Mapping)

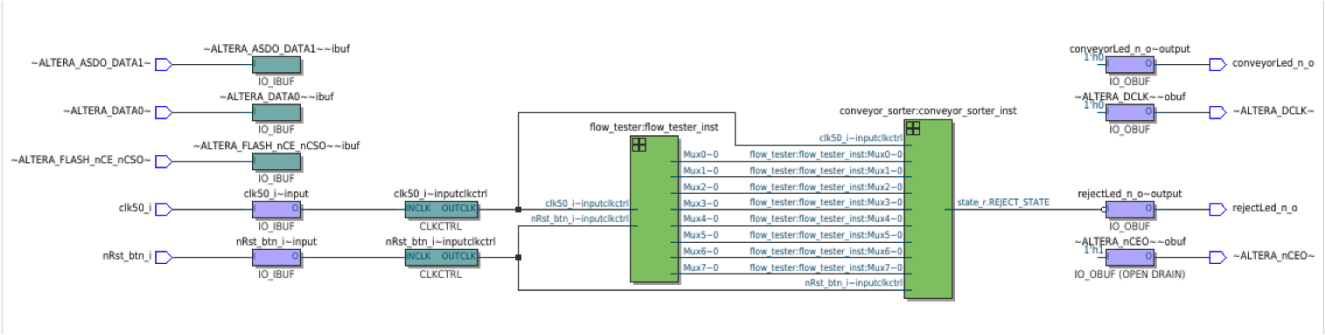


Рис. 8. Технологическая схема после размещения (Post-Fitting)

9. Результаты прошивки ПЛИС

В файле QSF указаны контакты clk50_i — PIN_G1, nRst_btn_i — PIN_P4, conveyorLed_n_o — PIN_V10, rejectLed_n_o — PIN_U9 и стандарт 3.3-V LVTTL. Приложенный снимок Pin Planner относится к предыдущей версии проекта. Назначения в QSF не сверены со схемой фактически используемой платы; повторная компиляция после правок не выполнялась.

При такте 50 МГц ожидается постоянный сигнал конвейера и импульс отбраковки 100 мс один раз в 3,2 с. Результаты загрузки конфигурации и испытания платы не предоставлены.

10. Ограничения и допущения

Вес — код 0 … 255 без заданных физических единиц. Изделия проходят последовательно, освобождая оба датчика. Без совместного срабатывания центр не фиксируется и измерение может не завершиться. Тайм-аут, фильтрация шумов и аварийная остановка отсутствуют. Первый спад после центра завершает измерение, даже если вызван шумом. Силовая часть отбраковки не описана. Демонстрационный генератор чередует два заданных профиля; реальные датчики и весы не подключены.

11. Заключение

Сортировщик содержит автомат с пятью состояниями, проверяет диапазон веса 95–105 и формирует импульс отбраковки для недопустимого изделия. Переходы заданы оператором `case`; регистры и выходы обновляются отдельными процессами. Тестбенч компилируется, однако успешное прохождение его проверок не подтверждено из-за сбоя ModelSim при запуске. Снимки Quartus показывают успешную сборку предыдущей версии; текущие исходники требуют повторной компиляции. Загрузка конфигурации и проверка платы не выполнялись.

12. Приложение

Полные VHDL-исходники, сценарий проверки и настройки проекта. Номера слева соответствуют исходным строкам; длинные строки перенесены при отображении.

Приложение 1. Файл conveyor_sorter.vhd

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity conveyor_sorter is
6      generic (
7          WEIGHT_MIN          : natural  := 95;
8          WEIGHT_MAX          : natural  := 105;
9          REJECT_PULSE_CYCLES : positive := 5_000_000
10     );
11     port (
12         clk_i         : in  std_logic;
13         nRst_i        : in  std_logic;
14         sensorA_i     : in  std_logic;
15         sensorB_i     : in  std_logic;
16         weight_i      : in  std_logic_vector(7 downto 0);
17         conveyorRun_o : out std_logic;
18         reject_o      : out std_logic
19     );
20 end conveyor_sorter;
21
22 architecture rtl of conveyor_sorter is
23
24     type state_type is (WAIT_STATE, MEASURE_STATE, CHECK_STATE,
   REJECT_STATE, WAIT_CLEAR_STATE);
25
26     signal state_r          : state_type;
27     signal previousWeight_r : std_logic_vector(7 downto 0);
28     signal measuredWeight_r : std_logic_vector(7 downto 0);
29     signal rejectCount_r    : natural range 0 to REJECT_PULSE_CYCLES - 1;
30     signal centerFlag_r     : std_logic;
31
32 begin
33
34     state_transitions : process(clk_i, nRst_i)
35     begin
36         if nRst_i = '0' then
37             state_r <= WAIT_STATE;
38         elsif rising_edge(clk_i) then
39             case state_r is
40                 when WAIT_STATE =>
41                     if sensorA_i = '1' or sensorB_i = '1' then
42                         state_r <= MEASURE_STATE;
43                     end if;
44                 when MEASURE_STATE =>
45                     if unsigned(weight_i) < unsigned(previousWeight_r)
46                        and (centerFlag_r = '1' or (sensorA_i = '1' and
   sensorB_i = '1')) then
47                         state_r <= CHECK_STATE;
48                     elsif sensorA_i = '0' and sensorB_i = '0' and
   centerFlag_r = '1' then
49                         state_r <= CHECK_STATE;
50                     end if;
51                 when CHECK_STATE =>
52                     if to_integer(unsigned(measuredWeight_r)) >=
   WEIGHT_MIN
53                        and to_integer(unsigned(measuredWeight_r)) <=
   WEIGHT_MAX then
54                         state_r <= WAIT_CLEAR_STATE;
55                     else
56                         state_r <= REJECT_STATE;
57                     end if;
58                 when REJECT_STATE =>
59                     if rejectCount_r = REJECT_PULSE_CYCLES - 1 then
60                         state_r <= WAIT_CLEAR_STATE;
61                     end if;
62                 when WAIT_CLEAR_STATE =>
63                     if sensorA_i = '0' and sensorB_i = '0' then
```

```
                        state_r <= WAIT_STATE;
                    end if;
            end case;
        end if;
    end process;

    data_registers : process(clk_i, nRst_i)
    begin
        if nRst_i = '0' then
            previousWeight_r <= (others => '0');
            measuredWeight_r <= (others => '0');
            rejectCount_r    <= 0;
            centerFlag_r     <= '0';
        elsif rising_edge(clk_i) then
            case state_r is
                when WAIT_STATE =>
                    previousWeight_r <= weight_i;
                    rejectCount_r    <= 0;
                    centerFlag_r     <= '0';
                when MEASURE_STATE =>
                    if sensorA_i = '1' and sensorB_i = '1' then
                        centerFlag_r <= '1';
                    end if;
                    if unsigned(weight_i) < unsigned(previousWeight_r)
                       and (centerFlag_r = '1' or (sensorA_i = '1' and
sensorB_i = '1')) then
                        measuredWeight_r <= previousWeight_r;
                    elsif sensorA_i = '0' and sensorB_i = '0' and
centerFlag_r = '1' then
                        measuredWeight_r <= previousWeight_r;
                    else
                        previousWeight_r <= weight_i;
                    end if;
                when CHECK_STATE =>
                    if to_integer(unsigned(measuredWeight_r)) < WEIGHT_MIN
                       or to_integer(unsigned(measuredWeight_r)) >
WEIGHT_MAX then
                        rejectCount_r <= 0;
                    end if;
                when REJECT_STATE =>
                    if rejectCount_r = REJECT_PULSE_CYCLES - 1 then
                        rejectCount_r <= 0;
                    else
                        rejectCount_r <= rejectCount_r + 1;
                    end if;
                when WAIT_CLEAR_STATE =>
                    null;
            end case;
        end if;
    end process;

    output_logic : process(all)
    begin
        conveyorRun_o <= '1';
        reject_o      <= '0';
        if state_r = REJECT_STATE then
            reject_o <= '1';
        end if;
    end process;

end rtl;
```

Приложение 2. Файл conveyor_sorter_top.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity conveyor_sorter_top is
    port (
        clk50_i       : in  std_logic;
        nRst_btn_i    : in  std_logic;
        sensorA_i     : in  std_logic;
        sensorB_i     : in  std_logic;
        weight_i      : in  std_logic_vector(7 downto 0);
        conveyorRun_o : out std_logic;
        reject_o      : out std_logic;
        conveyorLed_n_o : out std_logic;
        rejectLed_n_o   : out std_logic
    );
end conveyor_sorter_top;

architecture Structural of conveyor_sorter_top is

    signal resetSync_r      : std_logic_vector(1 downto 0);
    signal sensorASync_r    : std_logic_vector(1 downto 0);
    signal sensorBSync_r    : std_logic_vector(1 downto 0);

    signal weightMeta_r     : std_logic_vector(7 downto 0);
    signal weightSync_r     : std_logic_vector(7 downto 0);
    signal weightPrevious_r : std_logic_vector(7 downto 0);
    signal weightStable_r   : std_logic_vector(7 downto 0);

    signal nRst_s           : std_logic;
    signal conveyorRun_s    : std_logic;
    signal reject_s         : std_logic;

begin

    process(clk50_i, nRst_btn_i)
    begin

        if nRst_btn_i = '0' then

            resetSync_r <= (others => '0');

        elsif rising_edge(clk50_i) then

            resetSync_r(0) <= '1';
            resetSync_r(1) <= resetSync_r(0);

        end if;

    end process;

    nRst_s <= resetSync_r(1);

    process(clk50_i, nRst_s)
    begin

        if nRst_s = '0' then

            sensorASync_r    <= (others => '0');
            sensorBSync_r    <= (others => '0');
            weightMeta_r     <= (others => '0');
            weightSync_r     <= (others => '0');
            weightPrevious_r <= (others => '0');
            weightStable_r   <= (others => '0');

        elsif rising_edge(clk50_i) then

            sensorASync_r(0) <= sensorA_i;
            sensorASync_r(1) <= sensorASync_r(0);

            sensorBSync_r(0) <= sensorB_i;
            sensorBSync_r(1) <= sensorBSync_r(0);

            weightMeta_r <= weight_i;
            weightSync_r <= weightMeta_r;

            if weightSync_r = weightPrevious_r then
                weightStable_r <= weightSync_r;
            end if;
```

```
79
80              weightPrevious_r <= weightSync_r;
81
82          end if;
83
84      end process;
85
86      dut_inst : entity work.conveyor_sorter
87          generic map (
88              WEIGHT_MIN          => 95,
89              WEIGHT_MAX          => 105,
90              REJECT_PULSE_CYCLES => 5_000_000
91          )
92          port map (
93              clk_i         => clk50_i,
94              nRst_i        => nRst_s,
95              sensorA_i     => sensorASync_r(1),
96              sensorB_i     => sensorBSync_r(1),
97              weight_i      => weightStable_r,
98              conveyorRun_o => conveyorRun_s,
99              reject_o      => reject_s
100         );
101
102     conveyorRun_o <= conveyorRun_s;
103     reject_o      <= reject_s;
104
105     conveyorLed_n_o <= not conveyorRun_s;
106     rejectLed_n_o   <= not reject_s;
107
108 end Structural;
```

Приложение 3. Файл flow_tester.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity flow_tester is
    generic (STEP_CYCLES : positive := 5_000_000);
    port (
        clk_i     : in  std_logic;
        nRst_i    : in  std_logic;
        sensorA_o : out std_logic;
        sensorB_o : out std_logic;
        weight_o  : out std_logic_vector(7 downto 0)
    );
end flow_tester;

architecture rtl of flow_tester is
    signal stepCount_r : natural range 0 to STEP_CYCLES - 1;
    signal position_r  : natural range 0 to 15;
    signal rejectProduct_r : std_logic;
begin
    counter_registers : process(clk_i, nRst_i)
    begin
        if nRst_i = '0' then
            stepCount_r <= 0;
            position_r  <= 0;
            rejectProduct_r <= '0';
        elsif rising_edge(clk_i) then
            if stepCount_r = STEP_CYCLES - 1 then
                stepCount_r <= 0;
                if position_r = 15 then
                    position_r <= 0;
                    rejectProduct_r <= not rejectProduct_r;
                else
                    position_r <= position_r + 1;
                end if;
            else
                stepCount_r <= stepCount_r + 1;
            end if;
        end if;
    end process;

    output_logic : process(all)
        variable peakWeight_v : natural range 80 to 100;
    begin
        -- Start with a valid product; alternate profiles after each 16-step cycle.
        if rejectProduct_r = '1' then
            peakWeight_v := 80;
        else
            peakWeight_v := 100;
        end if;
        sensorA_o <= '0';
        sensorB_o <= '0';
        weight_o  <= (others => '0');
        case position_r is
            when 2 => sensorA_o <= '1'; weight_o <= std_logic_vector(to_unsigned(20, weight_o'length));
            when 3 => sensorA_o <= '1'; weight_o <= std_logic_vector(to_unsigned(peakWeight_v - 50, weight_o'length));
            when 4 => sensorA_o <= '1'; sensorB_o <= '1'; weight_o <= std_logic_vector(to_unsigned(peakWeight_v - 20, weight_o'length));
            when 5 => sensorA_o <= '1'; sensorB_o <= '1'; weight_o <= std_logic_vector(to_unsigned(peakWeight_v, weight_o'length));
            when 6 => sensorA_o <= '1'; sensorB_o <= '1'; weight_o <= std_logic_vector(to_unsigned(peakWeight_v - 2, weight_o'length));
            when 7 => sensorB_o <= '1'; weight_o <= std_logic_vector(to_unsigned(peakWeight_v - 30, weight_o'length));
            when 8 => sensorB_o <= '1'; weight_o <= std_logic_vector(to_unsigned(30, weight_o'length));
            when others => null;
        end case;
    end process;
end rtl;
```

Приложение 4. Файл qmtech_demo_top.vhd

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity qmtech_demo_top is
5      port (clk50_i : in std_logic; nRst_btn_i : in std_logic;
   conveyorLed_n_o : out std_logic;
       rejectLed_n_o : out std_logic);
6  end qmtech_demo_top;
7
8  architecture structural of qmtech_demo_top is
9      signal sensorA_s, sensorB_s, conveyorRun_s, reject_s : std_logic;
10     signal weight_s : std_logic_vector(7 downto 0);
11 begin
12     flow_tester_inst : entity work.flow_tester
13         generic map (STEP_CYCLES => 5_000_000)
14         port map (clk_i => clk50_i, nRst_i => nRst_btn_i, sensorA_o =>
   sensorA_s, sensorB_o =>
       sensorB_s, weight_o => weight_s);
15     conveyor_sorter_inst : entity work.conveyor_sorter
16         generic map (REJECT_PULSE_CYCLES => 5_000_000)
17         port map (clk_i => clk50_i, nRst_i => nRst_btn_i, sensorA_i =>
   sensorA_s, sensorB_i =>
       sensorB_s, weight_i => weight_s, conveyorRun_o => conveyorRun_s,
   reject_o => reject_s);
18     conveyorLed_n_o <= not conveyorRun_s;
19     rejectLed_n_o <= not reject_s;
20 end structural;
```

Приложение 5. Файл struct_tb.vhd

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity struct_tb is
6  end struct_tb;
7
8  architecture simulation of struct_tb is
9      constant CLOCK_PERIOD : time := 20 ns;
10     constant REJECT_CYCLES : positive := 3;
11     signal clk_s         : std_logic := '0';
12     signal nRst_s        : std_logic := '0';
13     signal sensorA_s     : std_logic := '0';
14     signal sensorB_s     : std_logic := '0';
15     signal weight_s      : std_logic_vector(7 downto 0) := (others =>
   '0');
16     signal conveyorRun_s : std_logic;
17     signal reject_s      : std_logic;
18 begin
19     dut_inst : entity work.conveyor_sorter
20         generic map (REJECT_PULSE_CYCLES => REJECT_CYCLES)
21         port map (clk_i => clk_s, nRst_i => nRst_s, sensorA_i =>
   sensorA_s, sensorB_i =>
       sensorB_s,
22                   weight_i => weight_s, conveyorRun_o => conveyorRun_s,
   reject_o => reject_s);
23
24     clock_generator : process
25     begin
26         while true loop
27             clk_s <= '0'; wait for CLOCK_PERIOD / 2;
28             clk_s <= '1'; wait for CLOCK_PERIOD / 2;
29         end loop;
30     end process;
31
32     stimulus_and_check : process
33         procedure tick is
34         begin
35             wait until rising_edge(clk_s);
36             wait for 1 ns;
37         end procedure;
38
39         procedure reset_dut is
40         begin
41             nRst_s <= '0';
42             wait for 1 ns;
43             assert conveyorRun_s = '1' and reject_s = '0'
44                 report "Reset must keep the conveyor enabled and reject
   inactive" severity
       failure;
45             tick;
46             nRst_s <= '1';
47             tick;
48         end procedure;
49
50         procedure check_product(target_weight : natural; expected_reject :
   std_logic) is
51         begin
52             weight_s <= std_logic_vector(to_unsigned(target_weight,
   weight_s'length));
53             sensorA_s <= '1';
54             sensorB_s <= '0';
55             tick;
56             sensorB_s <= '1';
57             tick;
58             weight_s <= std_logic_vector(to_unsigned(target_weight - 1,
   weight_s'length));
59             tick;
60             assert reject_s = '0'
61                 report "Reject must remain inactive during the check
   state" severity failure;
62             tick;
63             assert reject_s = expected_reject
64                 report "Product was classified incorrectly" severity
   failure;
65
66             if expected_reject = '1' then
67                 for cycle in 2 to REJECT_CYCLES - 1 loop
```

```
68                      tick;
69                      assert reject_s = '1'
70                          report "Reject pulse ended before the configured
    duration" severity
    failure;
71                  end loop;
72                  tick;
73                  assert reject_s = '0'
74                      report "Reject pulse exceeded the configured duration"
    severity failure;
75              else
76                  tick;
77                  assert reject_s = '0'
78                      report "Valid product entered the reject state"
    severity failure;
79              end if;
80
81              tick;
82              assert reject_s = '0'
83                  report "Reject must remain inactive while waiting for
    sensor release" severity
    failure;
84              sensorA_s <= '0';
85              sensorB_s <= '0';
86              tick;
87              tick;
88              assert conveyorRun_s = '1' and reject_s = '0'
89                  report "Sorter did not return to the waiting state"
    severity failure;
90          end procedure;
91      begin
92          reset_dut;
93          check_product(94, '1');
94          check_product(95, '0');
95          check_product(105, '0');
96          check_product(106, '1');
97          check_product(80, '1');
98
99          reset_dut;
100         weight_s <= std_logic_vector(to_unsigned(110, weight_s'length));
101         sensorA_s <= '1';
102         tick;
103         sensorA_s <= '0';
104         weight_s <= (others => '0');
105         tick;
106         tick;
107         assert reject_s = '0'
108             report "Measurement without center detection must not be
    completed" severity
    failure;
109
110         reset_dut;
111         weight_s <= std_logic_vector(to_unsigned(80, weight_s'length));
112         sensorA_s <= '1';
113         tick;
114         sensorB_s <= '1';
115         tick;
116         weight_s <= std_logic_vector(to_unsigned(79, weight_s'length));
117         tick;
118         tick;
119         assert reject_s = '1'
120             report "Out-of-range product must enter the reject state"
    severity failure;
121         nRst_s <= '0';
122         wait for 1 ns;
123         assert conveyorRun_s = '1' and reject_s = '0'
124             report "Asynchronous reset must stop rejection immediately"
    severity failure;
125         tick;
126         nRst_s <= '1';
127         sensorA_s <= '0';
128         sensorB_s <= '0';
129         tick;
130         assert reject_s = '0'
131             report "Sorter must remain reset after an interrupted reject
    pulse" severity
    failure;
132
133         report "All conveyor sorter checks passed" severity note;
134         wait;
```

```
    end process;
end simulation;
```

Приложение 6. Файл flow_tester_tb.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use STD.ENV.ALL;

entity flow_tester_tb is
end flow_tester_tb;

architecture simulation of flow_tester_tb is
    constant CLOCK_PERIOD : time := 20 ns;
    constant STEP_CYCLES : positive := 8;
    constant REJECT_CYCLES : positive := 3;
    signal clk_s : std_logic := '0';
    signal nRst_s : std_logic := '0';
    signal sensorA_s, sensorB_s, conveyorRun_s, reject_s : std_logic;
    signal weight_s : std_logic_vector(7 downto 0);
    signal rejectPulses_s : natural := 0;
begin
    clk_s <= not clk_s after CLOCK_PERIOD / 2;

    generator_inst : entity work.flow_tester
        generic map (STEP_CYCLES => STEP_CYCLES)
        port map (clk_i => clk_s, nRst_i => nRst_s,
                  sensorA_o => sensorA_s, sensorB_o => sensorB_s, weight_o
=> weight_s);

    sorter_inst : entity work.conveyor_sorter
        generic map (REJECT_PULSE_CYCLES => REJECT_CYCLES)
        port map (clk_i => clk_s, nRst_i => nRst_s,
                  sensorA_i => sensorA_s, sensorB_i => sensorB_s, weight_i
=> weight_s,
                  conveyorRun_o => conveyorRun_s, reject_o => reject_s);

    monitor : process
        variable previousReject_v : std_logic := '0';
        variable pulseCycles_v : natural := 0;
    begin
        wait until rising_edge(clk_s);
        wait for 1 ns;
        assert conveyorRun_s = '1' report "Conveyor must remain enabled"
severity failure;
        if nRst_s = '0' then
            rejectPulses_s <= 0;
            previousReject_v := '0';
            pulseCycles_v := 0;
            assert reject_s = '0' report "Reject must be inactive during
reset" severity
    failure;
        else
            if reject_s = '1' then
                if previousReject_v = '0' then
                    rejectPulses_s <= rejectPulses_s + 1;
                end if;
                pulseCycles_v := pulseCycles_v + 1;
            elsif previousReject_v = '1' then
                assert pulseCycles_v = REJECT_CYCLES
                    report "Reject pulse has incorrect duration" severity
failure;
                pulseCycles_v := 0;
            end if;
            previousReject_v := reject_s;
        end if;
    end process;

    stimulus : process
        procedure tick is
        begin
            wait until rising_edge(clk_s);
            wait for 2 ns;
        end procedure;

        procedure check_product(productIndex : natural) is
            variable expectedPeak_v : natural;
        begin
            if productIndex mod 2 = 0 then
```

```
                expectedPeak_v := 100;
            else
                expectedPeak_v := 80;
            end if;
            for cycle in 1 to 16 * STEP_CYCLES loop
                tick;
                if cycle = 5 * STEP_CYCLES then
                    assert sensorA_s = '1' and sensorB_s = '1'
                        report "Both sensors must be active at the peak"
severity failure;
                    assert to_integer(unsigned(weight_s)) = expectedPeak_v
                        report "Incorrect alternating weight profile"
severity failure;
                end if;
            end loop;
            assert rejectPulses_s = (productIndex + 1) / 2
                report "Only alternate, invalid products must be rejected"
severity failure;
            assert reject_s = '0' report "Reject must end before the next
product" severity
    failure;
        end procedure;
    begin
        tick;
        tick;
        nRst_s <= '1';
        -- Valid, invalid, valid: the next scheduled profile would be
invalid.
        for product in 0 to 2 loop
            check_product(product);
        end loop;
        -- Reset must restore the valid-first sequence.
        nRst_s <= '0';
        tick;
        tick;
        nRst_s <= '1';
        check_product(0);
        check_product(1);
        report "All alternating flow checks passed" severity note;
        stop;
        wait;
    end process;
end simulation;
```

Приложение 7. Файл test_alternating.do

```
onerror {abort}
set source_dir [file normalize [pwd]]
file mkdir [file join $source_dir simulation alternating]
cd [file join $source_dir simulation alternating]
if {![file exists work]} {vlib work}
vmap work work
vcom -2008 [file join $source_dir conveyor_sorter.vhd]
vcom -2008 [file join $source_dir flow_tester.vhd]
vcom -2008 [file join $source_dir struct_tb.vhd]
vcom -2008 [file join $source_dir flow_tester_tb.vhd]
vsim -onfinish stop work.struct_tb
set BreakOnAssertion 2
run 400 ns
quit -sim
vsim -onfinish stop work.flow_tester_tb
set BreakOnAssertion 2
run -all
```

Приложение 8. Файл qmtech_demo.qsf

Файл настроек взят из текущего проекта. Контакты и стандарт IO указаны в QSF, но требуют сверки со схемой фактически используемой платы и повторной компиляции.

```
set_global_assignment -name FAMILY "Cyclone 10 LP"
set_global_assignment -name DEVICE 10CL080YU484C8G
set_global_assignment -name TOP_LEVEL_ENTITY qmtech_demo_top
set_global_assignment -name VHDL_INPUT_VERSION VHDL_2008
set_global_assignment -name VHDL_FILE conveyor_sorter.vhd
set_global_assignment -name VHDL_FILE flow_tester.vhd
set_global_assignment -name VHDL_FILE qmtech_demo_top.vhd
set_global_assignment -name SDC_FILE qmtech_demo.sdc
set_location_assignment PIN_G1 -to clk50_i
set_location_assignment PIN_P4 -to nRst_btn_i
set_location_assignment PIN_V10 -to conveyorLed_n_o
set_location_assignment PIN_U9 -to rejectLed_n_o
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to clk50_i
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to nRst_btn_i
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to
conveyorLed_n_o
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to rejectLed_n_o

set_instance_assignment -name PARTITION_HIERARCHY root_partition -to | -
section_id Top
set_global_assignment -name PARTITION_NETLIST_TYPE SOURCE -section_id Top
set_global_assignment -name PARTITION_FITTER_PRESERVATION_LEVEL
PLACEMENT_AND_ROUTING
    -section_id Top
set_global_assignment -name PARTITION_COLOR 16764057 -section_id Top
```

Приложение 9. Файл qmtech_demo.qpf

```
QUARTUS_VERSION = "22.1std"
PROJECT_REVISION = "qmtech_demo"
```

Приложение 10. Файл qmtech_demo.sdc

```
create_clock -name clk50 -period 20.000 [get_ports {clk50_i}]
```